Deep Dive: Granular Instrumental Variables

Understanding the Optimal Weights Γ

Macro Lab Group — Technical Supplement

April 15, 2026

1 Recap: The Simple vs. The Full Model

In the simple model, we assume all firms respond identically to macro shocks. The GIV was simply:

$$z_t^{simple} = \sum (S_i - 1/N) y_{it} \tag{1}$$

This works because $S_i - 1/N$ is orthogonal to a constant factor (where every firm has $\lambda_i = 1$). However, when firms have heterogeneous sensitivities λ_i, the simple weights fail. We need Γ.

2 Intuition: Γ as "De-biased" Firm Size

2.1 The Simple GIV as a Special Case

Think of the simple GIV weights as $\Gamma_i^{simple} = S_i - \bar{S}$. We take the raw size and subtract the mean size. This "centers" the weights around zero, which kills any factor that hits everyone equally.

2.2 The Problem with Raw Size

If large firms are systematically more sensitive to interest rates (a common factor η_t), then the size-weighted outcome y_{St} will move whenever interest rates move. If we use y_{St} as an instrument, we are accidentally instrumenting with interest rate shocks, not idiosyncratic firm shocks.

2.3 The Logic of Γ

Γ is the part of firm size that is **unrelated** to macro-sensitivity.

- We ask: "How much of this firm's size is explained by its factor loading λ_i?"
- We subtract that "explained" part from the actual size.
- The result, Γ_i, is the "Pure Granular Weight."

If λ_i is completely uncorrelated with S_i, then Γ will look almost exactly like the simple GIV weights. If they are correlated, Γ "tilts" the weights to protect the instrument's validity.

3 The Math: Γ as a Projection Residual

Mathematically, we want to find weights Γ that are as close as possible to the size weights S (for power), but are orthogonal to the loadings λ (for validity).

We solve the following optimization problem:

$$\min_{\Gamma} \sum_{i=1}^{N} (\Gamma_i - S_i)^2 \quad \text{subject to} \quad \sum \Gamma_i = 0, \quad \sum \Gamma_i \lambda_i = 0 \tag{2}$$

The solution to this problem is equivalent to the **residual** of a linear regression. If we regress the vector of sizes S on a constant and the vector of loadings λ:

$$S_i = \alpha + \beta \lambda_i + \text{residual}_i \tag{3}$$

Then the optimal weight Γ_i is exactly that residual.

3.1 General Matrix Formula

If there are multiple factors (a matrix Λ), the formula is:

$$\Gamma = M_{\tilde{\Lambda}} S = (I - \tilde{\Lambda}(\tilde{\Lambda}'\tilde{\Lambda})^{-1}\tilde{\Lambda}')S \tag{4}$$

where $\tilde{\Lambda} = [\mathbf{1}, \Lambda]$ is the matrix of loadings plus a constant. $M_{\tilde{\Lambda}}$ is the "annihilator matrix" that projects S into the space orthogonal to the factors.

4 Python Implementation

The following code demonstrates how to calculate Γ using ‘numpy‘.

```
import numpy as np

def calculate_gamma(S, Lambda):
    """
    S: Vector of firm sizes (N x 1)
    Lambda: Matrix of factor loadings (N x K)
    """
    N = len(S)
    # Add a constant (ones) to the loadings to satisfy sum(Gamma) = 0
    ones = np.ones((N, 1))
    X = np.hstack([ones, Lambda])

    # Standard OLS Projection: Gamma = S - X(X'X)^-1 X'S
    # This is equivalent to the residuals of S regressed on X
    beta = np.linalg.inv(X.T @ X) @ X.T @ S
    prediction = X @ beta
    gamma = S - prediction

    return gamma

# Example: 5 firms, 1 factor
S = np.array([0.4, 0.3, 0.15, 0.1, 0.05])
Lambda = np.array([1.2, 0.8, 1.0, 1.1, 0.9]) # Factor loadings

gamma = calculate_gamma(S, Lambda.reshape(-1, 1))

print(f"Simple weights (S - mean): {S - 0.2}")
```

```
print(f"Optimal weights (Gamma):   {gamma}")
print(f"Sum of Gamma (should be 0): {np.sum(gamma):.4f}")
print(f"Gamma . Lambda (should be 0): {np.dot(gamma, Lambda):.4f}")
```

5 Summary

The transition from z^{simple} to z^{Γ} is a move from **demeaning** to **orthogonalizing**. In the simple model, we demean by subtracting $1/N$. In the full model, we orthogonalize by subtracting the projection of size onto macro-loadings.